Mathematical foundations

Dormant timebombs: adversarial attacks against collapsed code models

October 15, 2026

Every formula this project rests on, in one place, with a pointer to the code that implements it. Written for the authors and anyone joining the project: it assumes transformers, cross-entropy and gradient-based discrete search, and assumes nothing about this repository.

Three threads run through it. **Collapse** is a dynamical system on a trajectory of models. **Surrogates** approximate a point on that trajectory the attacker cannot observe. The **attack** is a constrained discrete optimization whose feasibility is governed by a measurable scalar, the security margin. The last two sections cover the poisoning line of work and a symbol table.

Contents

1 Setup and notation **2**

2 The collapse process **2**
2.1 The two recursions . . . 2
2.2 The data mixture . . . 3
2.3 Why truncated sampling drives collapse . . . 3

3 Surrogates for an unobserved generation **3**
3.1 Logit space: the tilt . . . 3
3.2 Weight space: LoRA alpha scaling . . . 4
3.3 Data space: stacked truncation . . . 4
3.4 Choosing the factor n . . . 4
3.5 The factor the attack wants is solvable in closed form . . . 4
3.6 The entropy-matched tilt . . . 5

4 The adversarial search **6**
4.1 The objective . . . 6
4.2 The sink-conditional objective . . . 7
4.3 The discrete step . . . 8

5 The verdict **8**
5.1 The hit rule . . . 8
5.2 Sampled majority . . . 8
5.3 Loss and survival . . . 9

6 The security margin **9**
6.1 Three quantities . . . 9
6.2 How the margins relate to the verdicts . . . 9
6.3 Headroom, and its limits . . . 9
6.4 The admissibility band . . . 10

7 The collapse map (poisoning) **10**

8 Symbol table **11**

9 Where each formula lives **11**

1 Setup and notation

A trajectory of models $M_0, M_1, \ldots, M_G$ is produced by recursive training. M_{base} is the pristine pretrained model the trajectory starts from.

symbol	meaning
M_{base}	the pristine base model, never fine-tuned here
M_g	the checkpoint after g rounds of recursive training
D_g	the corpus generated by M_g
θ_g	the parameters of M_g
$l_M(x)$	the logit vector M assigns at context x
$p_M(\cdot \mid x)$	the next-token distribution, $\text{softmax}(l_M(x))$
s	the adversarial suffix, a token sequence
w, r	the wrong (insecure) and right (secure) reference implementations
V, T	vocabulary size, sampling temperature

The **per-token cross-entropy** of a target string y under model M, given a prompt built with suffix s, is the quantity every loss below is written in:

$$\text{CE}_M(y \mid s) \;=\; \frac{1}{|y|} \sum_t -\log p_M(y_t \mid \text{prompt}(s),\, y_{<t})\,. \tag{1}$$

It is a *mean* over the target's tokens, so $|y| \cdot \text{CE}_M(y \mid s) = -\log p_M(y \mid \text{prompt}(s))$ is the sequence log-likelihood under teacher forcing. Implemented in `TargetModel.target_losses` (`run_attack.py`).

2 The collapse process

2.1 The two recursions

Generation g trains on what generation $g-1$ wrote. This is the **data recursion** and it is the collapse mechanism:

$$D_g \;\sim\; M_g(\,\cdot \mid \text{instructions}\,), \qquad \text{sampled at } (T, \text{top-}p, \text{top-}k) \tag{2}$$

$$M_{g+1} \;=\; \text{finetune}(\text{init}_g,\; D_g)\,. \tag{3}$$

The **weight recursion** is a separate choice. By default $\text{init}_g = M_{\text{base}}$: each generation trains a new adapter on the pristine base, the "replace" setting of the model-collapse literature, so only the data recurs. With `-recursive_init`, the ablation, $\text{init}_g = M_g$, so one LoRA adapter keeps being optimized and the number of optimizer steps applied to it grows linearly in g; at a high real-data fraction those steps are mostly the same human rows again and again, a memorisation route to degradation that self-training does not cause. The two produce different curves from the same data, which is why both are in the artifact names. Every result measured before 2026-10-15 is on recursive lineages.

2.2 The data mixture

`-real_data_fraction` interpolates the training corpus between self-generated and human text while holding the corpus *size* fixed, so the number of optimizer steps per generation is constant and the collapse curve stays attributable to composition rather than to budget:

$$\text{corpus}_g \;=\; (1-f)\cdot \text{sample}(D_{g-1}) \;+\; f\cdot \text{sample}(H), \qquad f\in[0,1],\; g\geq 1. \tag{4}$$

$f = 0$ is the pure replace regime. $f = 1$ is the **non-collapsing control**: same generation count, same seeds, same step count, no self-generated data at all, which is what separates "self-training moved the model" from "iterated fine-tuning moved the model". Generation 0 is never mixed, since it trains on human data by definition. Implemented in `mix_real_data` (`run_baseline.py`).

2.3 Why truncated sampling drives collapse

Sampling at temperature T with nucleus or top-k truncation is a **mode-sharpening** map on the output distribution. The exact statement that matters is about a pair of tokens. At temperature T the normalizer cancels in a ratio, so for any two tokens a, b at the same context:

$$\log p_T(a \mid x) \;-\; \log p_T(b \mid x) \;=\; \frac{l_a(x) - l_b(x)}{T} \qquad \text{(exact)}. \tag{5}$$

A log-odds gap is therefore scaled by exactly $1/T$ per sampling step. Below $T = 1$ that is amplification. Retraining on those samples carries the sharpened distribution into the next generation's weights, which is the mechanism Section 7 models as a linear map in logit space. Truncation (top-$p < 1$, top-k finite) compounds it by deleting the tail outright.

Note what this does *not* say: the sequence-level log-odds of two different strings does not scale exactly by $1/T$, because the per-step normalizers no longer cancel between two different continuations. Only the single-position decision scales exactly.

3 Surrogates for an unobserved generation

The threat model gives the attacker M_{base} and M_0 only. Every surrogate below approximates M_n from those two, by assuming the trajectory is locally a straight line and extending it by a factor n.

3.1 Logit space: the tilt

$$l_{\text{ex}}(x) \;=\; l_{\text{base}}(x) \;+\; n\,(l_0(x) - l_{\text{base}}(x))\,. \tag{6}$$

Computed in float32, **not clamped**. Softmax is shift invariant and is evaluated by subtracting the row maximum, so a large logit cannot overflow; clamping to a fixed range would instead tie every token above the ceiling and flatten everything below the floor, destroying exactly the ranking the extrapolation produces. In distribution space this is an **exponential tilt**:

$$p_{\text{ex}}(\cdot \mid x) \;\propto\; p_{\text{base}}(\cdot \mid x)^{\,1-n} \;\cdot\; p_0(\cdot \mid x)^{\,n}, \tag{7}$$

which is the same algebra as classifier-free guidance with guidance weight n. Two consequences the project uses. It is differentiable in the input embeddings through both models, so the adversarial search can optimize against it exactly as against a real checkpoint. And it is a *distribution*, not a checkpoint, so nothing is trained to produce it.

`extrapolate_logits` (`utils/extrapolation.py`) is the single definition, shared by the attack surrogate and the corpus generator so the two cannot drift.

3.2 Weight space: LoRA alpha scaling

The same first-order step taken in parameters rather than logits:

$$\theta_n \;\approx\; \theta_{\text{base}} \;+\; n\,(\theta_0 - \theta_{\text{base}})\,. \tag{8}$$

A LoRA layer adds $(\alpha/\text{rank})\,BA$ to a frozen base weight, so the whole fine-tuning delta is proportional to α. Since M_0 is one fine-tuning step from M_{base}, scaling α by n *is* the weight-space counterpart of the tilt, and it costs one model instead of two. `build_scaled_adapter` (`utils/extrapolation.py`).

3.3 Data space: stacked truncation

Collapse is driven by each generation resampling its own truncated output, so n generations stack n truncations. If one step keeps the top p_1 of the probability mass, then

$$p_n \;=\; p_1^{\,n}. \tag{9}$$

p_1 is *fitted* to reproduce the real M_0, not chosen. The schedule saturates toward a point mass as n grows, which is the degenerate fixed point the real process converges to; a schedule linear in n would instead run off the end of the simplex. `surrogate_top_p` (`utils/extrapolation.py`).

This surrogate is a *corpus-level* object. Its teacher-forced cross-entropy is identical to the base model's, because truncation changes sampling and not likelihood, so it cannot serve as a search proxy; the attack rejects it explicitly.

3.4 Choosing the factor n

Three answers to one question, and they disagree, which is itself data.

The indexing rule $n = g + 1$ names the generation but asserts nothing about fidelity.

The capability probe (`-sf auto`) walks a ladder of factors top down and takes the largest at which the surrogate still solves a required fraction of the clean tasks. It scans rather than bisects, because capability is not monotonic in n.

The calibration solves, per generation, for the factor whose tilted perplexity matches the real checkpoint's, by false position in log-perplexity:

$$\text{find } n(g) \text{ such that} \qquad \log \text{PPL}_{\text{ex}}(n(g)) \;=\; \log \text{PPL}_{\text{real}}(g). \tag{10}$$

Fitted on held-out human data, because the attacker needs the surrogate placed where generation g sits on data nobody trained on. Since the fit needs checkpoints the attacker does not have, a one-parameter law is fitted through the measured points and used to extrapolate:

$$n(g) \;=\; 1 \;+\; \text{scale} \cdot \ln(1 + g). \tag{11}$$

`calibrated_factor` (`utils/extrapolation.py`). Measured on this repo's 0.5B run the indexing rule and the calibration diverge by many orders of magnitude in perplexity, which is why the law exists.

3.5 The factor the attack wants is solvable in closed form

Perplexity calibration places the surrogate where generation g sits as a language model. An attack needs it to sit where generation g sits *at the security decision*, which is a different quantity. That one is solvable rather than fittable.

Because the tilt is affine in the natural parameters and the partition functions cancel in a difference of log-probabilities taken at **one** position, the surrogate's decision margin is exactly affine in the factor:

$$m(n) \;=\; (1 - n)\, m_{\text{base}} \;+\; n\, m_0. \tag{12}$$

Two consequences, both available from two forward passes and before any search:

$$n^* \;=\; \frac{m_{\text{base}}}{m_{\text{base}} - m_0}, \qquad \text{usable at all} \iff m_0 - m_{\text{base}} < 0. \tag{13}$$

A non-negative slope means every increase of the factor drives the surrogate *further* toward the secure sink, so no factor whatsoever makes it model that target as insecure. This is a guarantee from the geometry rather than a heuristic. Measured on the 0.5B, $f = 0.5$ lineage at the pristine model's decision point, `fetch_user` has $n^* = 2.25$ and `load_config` $n^* = 4.58$; the other three targets are unusable, `run_ping` because its boundary lies behind the base model, which already prefers the shell string.

Both anchors must be scored at the *same* prefix or the cancellation does not happen and the law does not hold. `run_margin.py -sink` therefore evaluates every model at the pristine model's own decision point, and `run_attack.py -surrogate_factor sink` reads the resulting table (`sink_factor`, `run_margin.py`).

3.6 The entropy-matched tilt

The factor n scales the *difference* of the two anchors, which sets the direction of travel but cannot set sharpness independently: as n grows, the tilt (7) tends to a point mass on the argmax of the likelihood ratio p_0/p_{base} and its entropy falls to zero. The real lineage goes the other way. On the 0.5B run the mean next-token entropy S_M (36) rises from 0.26 nats at the base model to 0.42 by generation 9. A tilt large enough to model a late generation's direction is therefore far too sharp to model its distribution, and this, not the direction, is the capability ceiling at which the surrogate stops writing valid code. A temperature restores the missing degree of freedom.

Fix a context x, write $l_n = l_{\text{base}} + n(l_0 - l_{\text{base}})$ for the plain tilt (6), and for an inverse temperature $\beta > 0$ let $p_\beta(l) = \text{softmax}(\beta l)$ and $\mathbb{H}(\beta; l) = -\sum_v p_\beta(l)_v \log p_\beta(l)_v$.

Definition. The entropy-matched tilt at factor n is

$$\tilde{l}_n(x) \;=\; \beta^*(x)\, l_n(x), \qquad \beta^*(x) \text{ the } \beta > 0 \text{ with } \mathbb{H}(\beta;\, l_n(x)) = \mathbb{H}(1;\, l_0(x))\,, \tag{14}$$

equivalently $\tilde{l}_n = l_n/\tau$ with $\tau = 1/\beta^*$. The target is the generation-0 anchor's entropy at the same context and under the same forbidden-token mask. It costs no extra forward pass, since l_0 is already in hand for the tilt itself.

Existence and uniqueness. Let S be the support of the row. Then

$$\frac{\partial \mathbb{H}}{\partial \beta} \;=\; -\beta \operatorname{Var}_{p_\beta}(l), \tag{15}$$

which is strictly negative unless l is constant on S. So $\mathbb{H}$ is strictly decreasing in β, running from $\log|S|$ as $\beta \to 0$ to 0 as $\beta \to \infty$, and any target in that open interval has exactly one root. The solve is a bisection on $\log\beta$ over $[-3\ln 10,\, 3\ln 10]$, i.e. $\tau \in [10^{-3}, 10^3]$, for 12 halvings, which leaves the temperature within 0.2%; a bracketed bisection on a monotone function cannot miss the root, and a target at or beyond an endpoint, such as an exactly uniform anchor, converges to the bracket edge instead of failing. Newton with the closed form (15) was tried first and rejected: the variance collapses exactly where the tilt concentrates, the step diverges, and on random logits it returned $\tau = 0.015$ where the truth was 3.0.

Rank preservation. β^* is a positive scalar per row, so it changes the order of no pair of tokens. The argmax, and therefore greedy decoding, is identical with and without matching. What changes is the mass behind the ranking, which is what sampling and the cross-entropy loss see.

Geometry. In the exponential family with natural parameter $\theta = l$, the plain tilt lies on the e-geodesic through p_{base} and p_0, parameterized by n. Multiplying θ by β moves along a different e-geodesic, the ray from the uniform distribution through p_n. Together the two operations span a two-parameter exponential family

$$p \;\propto\; p_{\text{base}}^{a}\, p_0^{b}, \qquad (a, b) = (\beta(1-n),\ \beta n)\,. \tag{16}$$

The plain tilt is the slice $a + b = 1$. Entropy matching drops that constraint and replaces it with the level-set constraint $\mathbb{H}(p) = \mathbb{H}(p_0)$: the matched surrogate is the intersection of the ray through p_n with the anchor's entropy contour. The factor chooses the ray, the temperature chooses the point on it.

Interaction with the affine law. At one decision position the partition functions cancel in any log-odds, and a positive scalar on the logits scales that difference, so

$$\tilde{m}(n) \;=\; \beta^*(n)\,\big[(1-n)\,m_{\text{base}} + n\,m_0\big]. \tag{17}$$

Its sign is the sign of (12), so the boundary factor n^* of (13) is unchanged by matching; only the magnitude of the margin, and with it the scale of the hinge terms in the objective, is rescaled. The sink-solved factor and entropy matching therefore compose without re-solving anything.

Gradient. τ is detached, so $\partial \tilde{l}_n / \partial l_n = \beta^*$ is treated as a constant and the true dependence of β^* on the suffix is dropped, exactly as for a normalization constant. This is deliberate: including it would let the optimizer satisfy the loss by shifting the sharpness rather than the direction, and the construction's point is that the factor owns the direction and the anchor fixes the sharpness.

Why the anchor's entropy. The attacker observes p_{base} and p_0 but not the generation under attack. Since the measured entropy keeps rising past the anchor, matching to $\mathbb{H}(p_0)$ under-estimates the late-generation entropy and the surrogate is still somewhat too sharp, but it is a floor rather than a point mass. Extrapolating the target as well, $\mathbb{H}^* = \mathbb{H}_{\text{base}} + n(\mathbb{H}_0 - \mathbb{H}_{\text{base}})$, is a one-line change that has not been made, because the measured entropy trajectory is not linear in the generation and would need the same check the calibration law (11) received.

Measured at $n = 2.25$ on the 0.5B, $f = 0.5$ lineage the plain tilt solves 1 of 5 clean tasks and the matched tilt 4 of 5, at no measurable change in wall-clock time, since the solve touches only logits that are already materialized. `entropy_matched_scale` and the `match_entropy` flag of `extrapolate_logits` (`utils/extrapolation.py`); enabled in the attack with `-entropy_match`.

4 The adversarial search

4.1 The objective

Let c be the model under attack (the real checkpoint, or a surrogate in transfer mode) and let H be the **hold set**: the models that must keep writing the secure implementation. H is the pristine baseline, plus the generation-0 anchor when it is held, so $|H|$ is 1 or 2.

$$\begin{aligned}
L(s) \;&= \mathrm{CE}_c(w \mid s) && \text{(1) break} && (18)\\
&+\; \nu\,\mathrm{relu}\Big(m_t - \big(\mathrm{CE}_c(r \mid s) - \mathrm{CE}_c(w \mid s)\big)\Big) && \text{(2) decide} && (19)\\
&+\; \lambda\,\frac{1}{|H|}\sum_{h\in H}\mathrm{relu}\big(m - \mathrm{CE}_h(w \mid s)\big) && \text{(3) hold away} && (20)\\
&+\; \mu\,\frac{1}{|H|}\sum_{h\in H}\mathrm{CE}_h(r \mid s) && \text{(4) hold toward} && (21)
\end{aligned}$$

Term by term.

(1) drives the insecure implementation's likelihood up on the model under attack.

(2) is the term that matches the loss to the verdict. (1) falling says the insecure code got *likelier*, not that it overtook the secure one, and with two models in H pulling the suffix back toward the secure code a suffix can improve (1) indefinitely without ever flipping the answer. (2) is a hinge on the target's own decision margin and goes slack once the insecure code leads by m_t nats per token. $m_t = 0$ disables it and recovers the previous objective.

(3) is a hinge, not gradient ascent: unbounded ascent on $\mathrm{CE}_h(w)$ diverges. It stops contributing once a hold model is already m nats away from the insecure code.

(4) anchors the hold models on the secure implementation, so that "did not emit the insecure string" cannot be satisfied by making them emit garbage instead.

The $1/|H|$ is load-bearing. Summing instead of averaging doubles the pressure to hold the moment the anchor is added, while the single break term stays as it is, so λ, μ and m silently stop meaning what they were calibrated to mean. Measured on one cell at a fixed budget, the summed objective reached a best total of 5.90 and found no hit where the averaged one hits at 3.39. `run_selective_attack.py` generalizes the same rule to an arbitrary hold group.

`ContrastiveGCG.objective` and `.combined_gradient` (`run_attack.py`).

4.2 The sink-conditional objective

Every cross-entropy in (18)–(21) is a mean over the whole reference implementation, a hundred-odd tokens of which two or three decide the verdict. Measured on the 0.5B, $f = 0.5$ lineage that mean fell from about 3 to about 0.8 nats per token while the surrogate's own output was unclassifiable in 75 of 75 checks: the suffix had pushed the model off the task rather than across the decision.

Let k be the length of the shared token prefix of r and w, and let r_k and w_k be the first differing spans of the two references (a pure insertion is extended by one shared token so both are non-empty). For any model M define

$$\mathrm{pref}_M(s) \;=\; \log p_M(w_k \mid s, r_{<k}) \;-\; \log p_M(r_k \mid s, r_{<k}), \tag{22}$$

$$\mathrm{prefix}_M(s) \;=\; \frac{1}{k} \sum_{t<k} -\log p_M(r_t \mid s, r_{<t}). \tag{23}$$

Both spans are teacher-forced on the same prefix, so (22) is a difference of two log-probabilities at one context: a decision margin, the quantity the static classifier reads and the quantity Section 3.5 measures. The sink-conditional objective is

$$\begin{aligned}
L_{\mathrm{sink}}(s) \;=\; & \zeta\, \mathrm{relu}\big(m_s - \mathrm{pref}_c(s)\big) && \text{target prefers the insecure span} && (24)\\
& +\; \kappa\, \mathrm{prefix}_c(s) && \text{and still writes the prefix} && (25)\\
& +\; \lambda\, \frac{1}{|H|} \sum_{h \in H} \mathrm{relu}\big(m + \mathrm{pref}_h(s)\big) && \text{each hold model prefers the secure span} && (26)\\
& +\; \mu\, \frac{1}{|H|} \sum_{h \in H} \mathrm{CE}_h(r \mid s) && \text{and keeps writing the secure code.} && (27)
\end{aligned}$$

Three properties. Under the logit surrogate pref_c is exactly affine in the factor, by the argument of Section 3.5, so each target's boundary n^* is the natural factor to search at, and `-surrogate_factor sink` now moves the surrogate to each target's own root (plus `-sink_offset`) when its search starts. The entropy-matching temperature of Section 3.6 is a positive scalar on the logits and scales pref without changing its sign. And m_s is in nats of a two-way log-odds, so it maps directly onto the sampled verdict: at temperature T the insecure span wins the two-way race with probability $\mathrm{sigmoid}(m_s/T)$, about 95% at $m_s = 2$, $T = 0.7$.

The full-sequence means are still computed and recorded, so `-max_wrong_loss` keeps its meaning. `ContrastiveGCG._sink_objective` and `._sink_gradient` (`run_attack.py`); result files carry `_sinkobj`.

4.3 The discrete step

The suffix is represented as a one-hot matrix $X \in \{0,1\}^{|s| \times V}$, and the input embeddings are $e = XE$ for embedding matrix E. The loss is differentiable in X, giving

$$G \;=\; \nabla_X L(s) \;\in\; \mathbb{R}^{|s| \times V}. \tag{28}$$

$G_{i,v}$ is a first-order estimate of the change in L from putting token v at position i. One GCG step is then:

1. for each position i, take the k tokens with the most negative $G_{i,\cdot}$ as candidates (implemented as the top-k of $-G$, with disallowed tokens set to $+\infty$ first so they can never be selected, which is how the non-ASCII and special-token constraints enter);
2. draw B candidate suffixes, each replacing n_{replace} positions uniformly from those sets;
3. discard candidates that do not survive a detokenize/retokenize round trip;
4. evaluate L **exactly** on all survivors and keep the arg min.

The gradient only proposes; the exact evaluation decides. The search is greedy coordinate descent in this sense and never accepts a worse candidate. Steps 1 and 3 are `sample_ids_from_grad` and `filter_ids` (`utils/gcg.py`).

Note that this makes the search itself **decoding-blind**: teacher-forced cross-entropy involves no sampling, which is why the data-space surrogate of Section 3.3 is useless here and why the verdict of Section 5 has to be imposed separately.

5 The verdict

5.1 The hit rule

Let $\text{status}_M \in \{\text{secure}, \text{insecure}, \text{unclassified}\}$ be the static classification of what model M emits for the adversarial prompt. A hit requires all three of:

$$\text{status}_c = \text{insecure} \qquad \text{the model under attack writes the vulnerability} \tag{29}$$

$$\text{status}_b = \text{secure} \qquad \text{the pristine baseline does not} \tag{30}$$

$$\text{status}_a = \text{secure} \qquad \text{the generation-0 anchor does not} \quad \text{(when held)} \tag{31}$$

The third condition is what makes the claim about *collapse* rather than about fine-tuning: without it, a suffix that breaks every descendant of the base model counts, and that is a statement about the first fine-tune, not about the trajectory.

5.2 Sampled majority

Each model is decoded k times at the deployment settings and charged with the **majority** status, ties resolved against the hit. If a single sample is insecure with probability q, the model is charged insecure iff

$$\text{Binomial}(k, q) \;>\; k/2. \tag{32}$$

So the sampled verdict is, to first order, a threshold at $q = 1/2$, where the greedy verdict is a threshold on the argmax. **The two verdicts are two thresholds on one underlying quantity**, which is the subject of Section 6 and the reason the project measures that quantity directly.

5.3 Loss and survival

Under ancestral sampling the probability of emitting the target exactly is

$$P(y) \;=\; \exp(-|y| \cdot \mathrm{CE}(y))\,. \tag{33}$$

So a target of 40 tokens at CE = 0.05 nats/token is emitted about 14% of the time per draw, and at CE = 0.02 about 45%. This is what `-max_wrong_loss` thresholds: it refuses to count a hit until the insecure code is likely enough to survive being sampled, instead of accepting the first argmax flip. Truncation raises these numbers, since mass deleted from the tail is redistributed.

6 The security margin

A decoding-independent scalar that both verdicts are thresholds on. Measured on the **clean** prompt to characterize a model, and on the adversarial prompt to characterize a suffix. Implemented in `measure()` (`run_margin.py`).

6.1 Three quantities

Let k be the first position at which the tokenizations of r and w diverge, with the shared prefix teacher-forced.

Decision margin, the log-odds of the secure token over the insecure one at the branch point:

$$\mathrm{dm}_M \;=\; \log p_M(r_k \mid \text{prefix}) \;-\; \log p_M(w_k \mid \text{prefix})\,. \tag{34}$$

Sequence margin, the same at the level of whole implementations:

$$\mathrm{sm}_M \;=\; \log p_M(r) - \log p_M(w) \;=\; |w| \cdot \mathrm{CE}_M(w) \;-\; |r| \cdot \mathrm{CE}_M(r). \tag{35}$$

Entropy, the mean next-token entropy over the secure target's positions, which is a direct read-out of the sharpening of Section 2.3 and serves as the collapse-severity axis:

$$S_M \;=\; \frac{1}{|r|} \sum_t \mathbb{H}(p_M(\cdot \mid r_{<t}))\,. \tag{36}$$

6.2 How the margins relate to the verdicts

Greedy decoding emits the secure token at the branch point iff it is the argmax, which requires $\mathrm{dm}_M > 0$ but is not implied by it, since a third token may lead. The measurement records `secure_is_argmax` separately for exactly this reason, and it is often false: the model phrases the function differently from both references, in which case dm compares two tokens the model was never going to write and sm is the quantity to read.

By the exact result of Section 2.3, the decision margin at temperature T is dm/T. So lowering the temperature scales the decision margin up and makes a flip harder, and raising it scales toward zero and makes the outcome a coin flip. This is the formal content of "exploitability is decoding-conditional".

6.3 Headroom, and its limits

For the three-sided hit rule to be satisfiable, the model under attack must be *closer* to the insecure variant than the anchor it has to spare. Hence

$$\mathrm{headroom}(g, \text{task}) \;=\; \mathrm{sm}_{M_0}(\text{task}) \;-\; \mathrm{sm}_{M_g}(\text{task}). \tag{37}$$

Positive headroom means collapse has already moved the target in the attack's favour relative to the model that must be held. Negative headroom means the search is asked to drag the target past a model it must simultaneously spare.

Headroom is a prior, not a predictor. Measured on the $f = 0.5$ lineage, the only cell that produced hits was ranked last of its peers by both the sequence and the decision form. A clean-prompt margin says where a model starts, not how far a suffix can move it; the quantity that would predict success is a sensitivity, not a level. Use the ranking to report and to diagnose, not to filter.

6.4 The admissibility band

Two conditions must hold at once for a cell (g, task) to support the claim. The capability gate requires M_g to write *secure* code unaided, and headroom requires M_g to be *closer to insecure* than M_0. These pull in opposite directions, and their intersection is the real object of study. Measured on the $f = 0.5$ lineage over 45 cells:

cells	count
total	45
pass the capability gate	14
positive headroom	13
both	3

The clearest illustration is the OS-command-injection target, which has the largest headroom at every generation and is inadmissible at every generation because the anchor already writes the vulnerability with no suffix at all.

7 The collapse map (poisoning)

The poisoning line models collapse as a map on a scalar **observable** rather than on weights. Let r_n be the trigger-context payload rate at generation n: the fraction of answers to trigger-bearing instructions that contain the payload. One generation resamples and retrains, which by Section 2.3 is near-linear in logit coordinates:

$$\operatorname{logit}(r_{n+1}) \;=\; a \cdot \operatorname{logit}(r_n) \;+\; b. \tag{38}$$

with a the sharpening slope. Section 2.3 predicts $a \approx 1/T$, about 1.43 at $T = 0.7$, but a is *measured*, never assumed. The map is affine, so it has a single fixed point

$$L^* \;=\; \frac{b}{1-a}, \qquad r^* \;=\; \operatorname{sigmoid}(L^*). \tag{39}$$

In the shifted coordinate $u_n = \operatorname{logit}(r_n) - L^*$ the map is pure geometric growth, $u_n = a^n u_0$, giving three closed forms:

$$\text{forecast} \qquad r_n \;=\; \operatorname{sigmoid}\left(L^* + a^n \left(\operatorname{logit}(r_0) - L^*\right)\right) \tag{40}$$

$$\text{activation} \qquad n^* \;=\; \frac{\log\left(-L^* / \left(\operatorname{logit}(r_0) - L^*\right)\right)}{\log a} \tag{41}$$

$$\text{required dose} \qquad r_0 \;=\; \operatorname{sigmoid}\Big(L^* \left(1 - a^{-n^*}\right)\Big) \tag{42}$$

where activation is the generation at which greedy decoding flips, i.e. $\operatorname{logit}(r) = 0$.

The fixed point is unstable iff $a > 1$, and a dormant payload amplifies iff additionally $r^* < 0.5$, so that the initial rate sits on the growing side. When those fail, no generation-0 dose

arms the fuse and the tools return infeasible rather than a number. That is a finding about the attack surface, not a tuning failure: the LoRA fine-tune damps the sharpening.

Implemented in `utils/collapse_map.py`, derived in `collapse_map_poison.md`.

The earlier gradient-matching variant optimizes poison content against the *weight* surrogate of Section 3.2. If a reference backdoor sits at θ^*, then under $\theta_k \approx \theta_{\text{base}} + k(\theta_0 - \theta_{\text{base}})$ the generation-0 delta needed to arrive there at generation k is

$$\Delta W_0 \;=\; \frac{\Delta W^*}{k}, \tag{43}$$

and the poison text is searched to make the training gradient point at it, by minimizing $-\cos(\nabla_B L,\, B^*)$ with the LoRA basis frozen. See `gradient_matching_poison.md`. It assumes weight-space collapse, which is not the regime this pipeline is in, which is why the data-space map above replaced it.

8 Symbol table

symbol	meaning	default
n	surrogate extrapolation factor	$g+1$
n^*	factor at which the surrogate's decision margin is 0, eq. (13)	solved
β^*, τ	entropy-matching inverse temperature and temperature, eq. (14)	solved
f	real-data fraction in the training mixture	0
T, p, k	sampling temperature, nucleus, top-k	0.7, 0.8, 20
λ	weight of the hold-away hinge	1.0
μ	weight of the hold-toward term	0.5
m	hold-away hinge margin	3.0
ν, m_t	weight and margin of the target-side hinge	1.0, 0 (off)
ζ, m_s, κ	sink-conditional break weight, margin, stay-on-task weight	1.0, 2.0, 1.0
B, k_{top}	GCG search width, gradient top-k	256, 256
H	hold set, {baseline} or {baseline, anchor}	with anchor
a, b, L^*	collapse-map slope, intercept, fixed point	measured
r_n	trigger-context payload rate at generation n	measured

9 Where each formula lives

formula	file
per-token cross-entropy, objective, gradient	`run_attack.py`
sink-conditional objective, sink spans	`run_attack.py`
GCG candidate step	`utils/gcg.py`
logit tilt, alpha scaling, top-p schedule, calibration law	`utils/extrapolation.py`
entropy-matched tilt	`utils/extrapolation.py`
affine law, boundary factor	`run_margin.py`
data mixture	`run_baseline.py`
margins, entropy, headroom	`run_margin.py`
sampled majority verdict	`run_attack.py`
collapse map and its closed forms	`utils/collapse_map.py`